

VRIJE
UNIVERSITEIT
BRUSSEL



Graduation thesis submitted in partial fulfilment of the requirements for the degree of de Ingenieurswetenschappen: Computerwetenschappen

PROGRAMMING ROBOTS WITH HIGHER-ORDER REACTIVE PROGRAMMING

Gérard Lichtert

2026-2027

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Sciences and Bio-engineering Sciences

VRIJE
UNIVERSITEIT
BRUSSEL

Proefschrift ingediend met het oog op het behalen van de graad van Master of Science in de Ingenieurswetenschappen: Computerwetenschappen

ROBOTS PROGRAMMEREN MET HOGERE-ORDE REACTIEF PROGRAMMEREN

Gérard Lichtert

2026-2027

Promotor: Prof. Dr. Wolfgang De Meuter
Advisor: Bjarno Oeyen

Wetenschappen en Bio-ingenieurswetenschappen

Abstract

Reactive Programming (RP) is a declarative programming paradigm based on data streams and the propagation of change. In a nutshell, if we have two variables x and y and another variable z that sums both x and y, it should be sufficient to update either x or y and z should be updated automatically.

There are several approaches to achieve Reactive Programming. One approach is to create a dedicated language. In this case the reactivity will be native to the language. Another option is to extend a language with reactive features, like

Contents

1 Introduction **1**

1.1 Contributions 2

1.2 Overview of this Dissertation 3

1.3 Approach 3

2 State of the Art **5**

2.1 Reactive Programming 6

2.1.1 Hosted and independent reactive languages 7

2.2 RP languages targeting MCUs 9

2.2.1 EmFRP 10

2.2.2 CFRP 11

2.2.3 Haai + Remus 13

2.3 Robotics frameworks 13

2.3.1 ROS 14

2.3.2 OROCOS 20

2.3.3 YARP 24

2.4 Revisiting the Problem Statement 26

2.5 Tierless Programming 28

3 μ-Remus **31**

3.1 Haai & Remus 31

3.2 ROS & μ-ROS 31

3.3 Mapping RP to ROS & μ-ROS 31

3.4 μ-Remus: A C implementation 31

4 Programming Robots with Reactors **33**

4.1 Linking ROS and μ-ROS 33

5 Tierless Haai **35**

5.1 Haai to CHaai 35

6 Evaluation **37**

6.1 Evaluating Haai distribution 37

6.2 Evaluating Haai to CHaai 37

6.3 Evaluating CHaai on μ-Remus 37

7 Conclusion & Future Work **39**

Chapter 1

Introduction

Modern software is often written in such a way that it is event-driven, or in other words, reactive. This means that applications reacts to an incoming event by creating subsequent events that arise as a reaction to the incoming event. Such an application is often called a reactive system. Reactive systems are often implemented with callbacks or by implementing the Observer Pattern. However, this does not come without its drawbacks as this may lead to 'inversion of control' or 'callback hell'[1], [2]. To alleviate this we introduce **Reactive Programming**. A programming paradigm that allows the user to write reactive systems in a declarative way. Traditionally the sum and the product of two elements would be written as shown in Listing 1.1.

```
int x = 5;
int y = 10;
int s = x + y;
int p = x * y;
// x changes state
x = 10;
// s requires recomputation
s = x + y;
// p requires recomputation
p = x * y;

```

Listing 1.1: Imperative product and sum

Whenever x or y would change state, you would have to manually update s and p. Reactive Programming allows you to declare x and y as sources of data and s and p as the result of the relevant computations. Whenever the value of x or y would change, the variables dependent on the sources would be automatically recomputed, and the result propagated further in the reactive system. Schematically the dependency graph of product-sum would look like Figure 1.1 There are different ways of how such a reactive system could be created using Reactive Programming. One way is to use a programming which itself is natively reactive. A few examples are Haai [3], Another is to add native reactivity to an existing language by

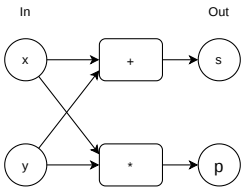


Figure 1.1: Dependency graph for product-sum

extending it the language or by implementing a reactive library such as ReactiveX. We will be looking at the former and more specifically the higher-order reactive programming language **Haai**, which was developed at the SOFT lab at the VUB. Haai uses reactors as their As a preview, an example of how such a product and sum could be written in Haai is shown in Listing 1.2. Haai uses reactors as its native way of declaring a reactive program.

```
(defr (product-sum x y)
    (def sum (+ x y))
    (def product (* x y))
    (out product sum))

```

Listing 1.2: Product sum in Haai

Additionally, we want to delve into the world of electronics. From maintaining your fridge's temperature to controlling a drones' altitude, electronics are deeply embedded in everyone's day-to-day life. Writing software for these electronics can also be challenging. This mainly due to the fact that using peripherals require using their associated libraries, which are not always standardized. Another challenge is that creating programs for microcontrollers often require the developer to use a low-level programming language such as C/C++ or Rust, which come with their own challenges. Nowadays, there are some efforts being made to provide a standardized collection of tools to aid with the development of systems with many peripherals, or robots. There is Robot Operating Systems (ROS) [4], Open Robot Control Software (OROCOS) [5], Yet Another Robot Platform (YARP) [6] to name a few. We will be focussing on using **(ROS)**.

1.1 Contributions

Reactive Programming is used across a multitude of domains. It is used in web and mobile development for instance. However, we will be focussing on using it for the Internet of Things (IoT). IoT consists of multiple physical objects, usually with very limited computing power and/or memory. In this thesis we introduce a way of declaring a reactive program across multiple devices from a single source. This means that the developer declares the program once, along with declarations where which part of the program lives and then deploys the program to the relevant devices making use of ROS. This is achieved by the following components:

- An extension of the Remus VM [7], more specifically, extending the language to accept a declaration of where which part of the program lives. The Remus compiler will use this metadata to split the Haai reactor declarations into an embedded part and a server part. Each piece will then be available for the user to be deployed in their respective parts of the IoT network.
- A new tool called CHaai, which translates the Remus bytecode obtained from compiling the Haai program into a C program. The C program will be required to run the program on microcontrollers using ROS.
- A C port of the existing Remus VM written in rust [8], also created to allow developers to create reactive programs for microcontrollers. However, it is adapted with respects to the current Instruction Set Architecture (ISA) of the Racket version of Remus [7] and allows usage of ROS.

1.2 Overview of this Dissertation

Chapter 2 gives the problem statement as well as an overview of the current Reactive Programming solutions and existing toolkits to program networks of microcontrollers. It also introduces a concept called 'Tierless programming', which will inspire the declarative way of telling Remus where which part of the reactive program lives.

Chapter 3 introduces μ-Remus. A C port of the Remus VM, which was previously written in Rust as well as in Racket. It explains why the C port was necessary as well as conceptualizes the link between the ROS communication protocols and the dependency graph generated by Reactive Programming.

Chapter 4 demonstrates usage of the Haai language extensions inspired by Tierless programming to declare where which part of the reactive program lives. It will also tell you about the CHaai tool. Which will create a C version of the reactive program. It is ported to C to be compatible with ROS.

Chapter 5 will evaluate the tools and extensions driven by an example. The target microcontroller will be a Raspberry Pi Pico H (2021) and will be used along a PC.

Lastly, chapter 6 will conclude this dissertation and give notes on some possible future work.

1.3 Approach

We first start by discussing the state of the art, what tools already exist and then move on to the problem statement and continue with the contributions part of the dissertation. However, since it is heavily dependent on a language and a toolkit we will first give a brief introduction into some relevant parts of the language or toolkit, each time providing the necessary context and information, prior to moving on to the actual contributions.

Chapter 2

State of the Art

Many programs spend most of their time reacting to events or their environment: a thermostat displays the temperature a sensor picks up, a lightbulb reacts to the pressing of a button, a notification is pushed once an email arrives. Reactive Programming (RP) is a programming paradigm that suits these kinds of programs. Instead of writing code that explicitly updates values when something changes, the programmer declares how values depend on one another and the language takes care of propagating changes through those dependencies [1].

Embedded devices are prime examples of devices that spend their lifecycle reacting to their surroundings, often taking part in a much larger system. Consider, for instance, a modern-day car, which is equipped with a myriad of subsystems. A braking system reacts to an obstacle drawing too close to the front of the car while a dashboard behind the steering wheel reacts to the metrics fed by other subsystems such as temperature, speed, gear shift, etc.... All these subsystems communicate with each other and depend on one another to form a much larger, coherent system. Inherently, these systems are distributed as they span multiple devices. Robots are no different: their sensors, actuators and controllers often run on separate boards that must also cooperate.

Programming such a system is not trivial. Each device runs its own program, written for its hardware. Additionally, the developer has to connect these devices through a medium that allows them to communicate with each other. While it is possible to write reactive programs for all these devices and then glue them manually, doing so is cumbersome [1], [9]. Creating a single RP program and avoiding this manual glue is the challenge of this dissertation. Moreover, we want to build on an existing robotics framework, so that its software tools and libraries can aid the development of future robotics software. The motivation behind choosing RP as our paradigm is given together with the necessary background in Section 2.1.

The following requirements arise. First, we need to choose an RP language that runs on microcontrollers (MCUs). This is discussed in Section 2.2. Second, we need a robotics framework that has an SDK and a user base, and runs on MCUs. Preferably, it also abstracts away the communication between devices, independent of the medium used (e.g. UART, Bluetooth). This is discussed in Section 2.3. Third, the robotics framework has to work

together with the RP language. This is discussed in Section 2.4.

2.1 Reactive Programming

Software nowadays is largely event-driven, meaning that programs spend most of their time reacting to incoming data. In imperative languages this is traditionally implemented with handlers, callbacks or the Observer pattern. The programmer registers a function with an event source and the runtime calls that function when that event occurs [1].

Unfortunately, this scales poorly as it gives rise to two problems: callback hell and inversion of control [1]. Callback hell occurs when callbacks are so deeply nested that it becomes difficult to tell what each callback does and when it is invoked. An example is shown in Listing 2.1, where a temperature sensor reads its pin, normalizes the read voltage into temperature and displays it on a screen. Any errors while displaying or sending are logged.

```
readSensor((voltage) => {
    normalize(voltage, (temperature) => {
        display(temperature, (errors) => {
            log(errors, () => {
                sendToServer(temperature, (errors) => {
                    log(errors, () => {
                        // etc...
                    })
                })
            })
        })
    })
})
```

Listing 2.1: Illustration of callback hell in JavaScript

Inversion of control is related to callbacks: instead of the programmer's code being in charge, the programmer hands the function to a framework or event source and the framework decides when it is called. As a result, the control flow is no longer visible in the program text as the framework decides how and when the programmer's code is run. In Listing 2.2 we define two functions `hello` and `foo` within the Flask web framework. These functions are registered to be called when their respective endpoints `"/hello"` and `"/foo"` are requested. The programmer never calls them directly. Additionally, the flow between receiving the request and the registered function being called is hidden inside the framework.

Reactive Programming (RP) attempts to solve these problems [1]. RP is a declarative programming paradigm that allows the developer to declare a program in terms of dependencies, instead of having to manually manage these dependencies [1]. A value that is computed from other values is said to be dependent on them. When these values in turn also have other values that they depend on, they form a dependency chain. In an imperative language, you would have to manually update all these values in the correct order. In RP, the

```python
@app.route("/hello")
def hello():
    return "world"

@app.route("/foo")
def foo():
    return "bar"
```

Listing 2.2: Inversion of control in Python, using the Flask web framework

actual propagation of the state change in the dependency chain is abstracted away in the language design. For instance, in a program you could have two variables `x` and `y` that hold values and variable `s` which holds the sum of `x` and `y`. Under the imperative paradigm, whenever `x` or `y` would change state, it would require manual recomputation of `s` as shown in Listing 2.3.

```c
int x = 5;
int y = 10;
int s = x + y;
printf("%d", s); // 15
x = 10; // state change
printf("%d", s); // 15
// requires manual recomputation of s
s = x + y;
printf("%d", s); // 20
```

Listing 2.3: Manual recomputation of `s` in C

In Reactive Programming, it is sufficient to update either `x` or `y` and the change is propagated throughout the dependency chain. A dependency chain can also be seen as a directed acyclic graph (DAG), with input variables and output variables, often called *signals* [1]. Converting Listing 2.3 into a DAG yields the graph shown in Figure 2.1. In Listing 2.3, the programmer has to maintain these dependencies manually; in RP, the language does so automatically. In essence, whenever an input signal changes state, the change is propagated through the graph and finally updates its output signals. An example in the reactive language REScala is shown in Listing 2.4. The input signals `x` and `y` are declared using the `Var` class and the dependent signals can be derived with `Signal { ...}`, as seen in the example with `sum`. In this example we can see that `s` is declared once through the `Signal` dependency, instead of nested callbacks. Whilst RP does not fully solve inversion of control, since the runtime still decides when it is called, it does make the data flow explicit [10].

2.1.1 Hosted and independent reactive languages

In practice there are two kinds of reactive languages. The first kind are *hosted* reactive languages, which are built on top of an existing host language. Examples are REScala, built on

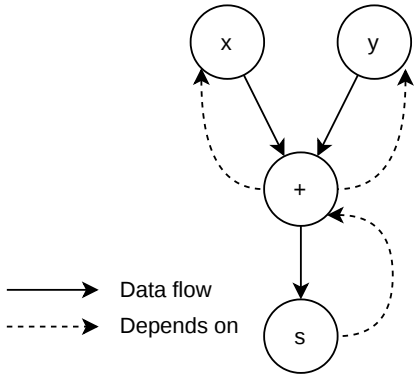


Figure 2.1: Dependency graph for `x`, `y` and `s`

```scala
def sum(a: Signal[Int], b: Signal[Int]): Signal[Int] = Signal { a.value + b.value}

@main def demo(): Unit =
    val x = Var(5)
    val y = Var(10)
    val s = sum(x, y)

    println(s"s = ${s.now}") // s = 15
    x.set(10)
    println(s"s = ${s.now}") // s = 20
```

Listing 2.4: Automatic recomputation of `s` in REScala

top of Scala, and Yampa, built on top of Haskell. These RP languages inherit the tools of their host language such as compilers, runtime, and libraries [1]. These reactive languages consist of two parts: a reactive part and a non-reactive part of the code. The reactive part is the part that the host language is extended with. The non-reactive part is functions or code of the host language that work on signals through a mechanism called *lifting*. However, lifting has two drawbacks. First, it can lead to *reactive thread hijacking*: the runtime has to wait for the lifted function to return, so a function that never returns takes over the runtime and makes the program unresponsive. Second, lifting can cause a *reactive/imperative impedance mismatch*: the runtime, not the programmer, decides the order in which the lifted functions run, so side effects such as I/O may happen in an unintended order and lead to faulty behaviour [3].

The second kind are *independent* reactive languages. These languages were built from the ground up with their own compilers, runtime, tooling, etc. Some examples are Haai, EmFRP, CFRP and Elm [1]. This does not mean that they avoid lifting, since some of these languages still use a non-reactive part to feature data structures and functions within the reactive part [1]. An independent reactive language lacks the ecosystem of a host language, and reusing existing code requires a foreign function interface. In return, its designers control the entire implementation, so they can enforce the properties their target domain needs, such as safety and efficiency [1].

An extreme case is *Haai*, a pure reactive programming language, which has no non-reactive parts [3]. In Haai a reactor is the only computational unit, which is essentially a description of the DAG shown in Figure 2.1. Instantiating a reactor is called a *deployment*, which supplies the reactor with its concrete input signals, called *sources*, and creates its output signals, called *sinks* [3]. The benefit of Haai is that everything is a reactor; even primitives such as `+` and `-` are reactors, and not functions. This means that there is no lifting and consequently Haai does not suffer from the drawbacks of lifting (thread hijacking and impedance mismatch) [3]. An example of a Haai program can be seen in Listing 2.5. `defr` is used to declare a reactor along with its name and its sources `x` and `y`. `def` is used to define internal signals and `out` is used to declare one or multiple sinks, though if there is only one sink, ending the reactor with it is sufficient instead of using `out` [3]. In Listing 2.5 we define `s` as an internal signal, which is the result of the deployment of the `+` reactor. The signal `time` is a built-in signal that increases its value over time. Should the values of the sources `x` or `y` change, the change is automatically propagated to the deployment of the `+` reactor and to the sink `s`. In the case of `total`, which is the result of the deployment of the `sum` reactor with the sources `time` and `10`, the sum is recomputed each time the value of the `time` signal changes.

```
(defr (sum x y)
    (def s (+ x y))
    (out s))

(def total (sum time 10))
;; total = 10, 11, 12, ...
```

Listing 2.5: Reactive program in Haai

2.2 RP languages targeting MCUs

Reactive Programming (RP) has many uses, from web applications that react to the input of users to microcontrollers (MCUs), which react to signals from hardware [1], [11]. In this thesis we focus on RP for MCUs, which are used in a growing number of domains, such as the Internet of Things (IoT) [12], edge computing [13] and artificial intelligence (AI) [14].

MCUs have little memory and processing power, and support few programming languages, mostly low-level ones such as C, C++ and Rust [15]. This is also why it is hard to use hosted reactive languages on these devices: they often rely on the runtime of their host language [1], [7]. For instance, Yampa, a language built on top of Haskell, cannot be used on most MCUs because the Haskell runtime is too large to be flashed onto them [16]. Despite this, there have been efforts to get reactive languages running on MCUs.

To compare these languages, we use the following requirements:

- *Predictable memory footprint*: the program's memory usage must be known in advance to ensure it meets the memory constraints of the MCU.

- *Access to sensors and actuators*: the program must be able to read sensors and drive actuators, either through the language itself or through MCU-compatible code.
- *Interoperability*: the reactive language must be able to interoperate with a low-level language that the robotics framework is written in.
- *Portability*: the reactive program must be able to run on different MCUs within a system, or robot.

Table 2.1 gives a high-level overview of independent RP languages that were designed to run on MCUs, and whether they meet the requirements above.

Some honourable mentions that fell just short of Table 2.1 are Hailstorm, ReactiFi and RT-FRP. Hailstorm runs on top of the Erlang virtual machine, which occupies about 30 MB of memory [17]. Like the Haskell runtime, this exceeds the memory of most MCUs. Additionally, running Hailstorm on MCUs was left as future work, and in the presence of recursion its memory usage cannot be predicted statically [17]. ReactiFi specifically targets Wi-Fi chips [8], [18], so unlike the other languages in Table 2.1, it is not a general-purpose language for MCUs. RT-FRP statically bounds the time and memory of each execution step [19], but its focus is on the formal integration with a base language rather than on implementing the language. Its successor E-FRP, which is included in the table, shifts the focus to implementation and was compiled to run on an MCU [20].

Language	Predictable memory	Sensors & actuators	Interoperability	Portability
EmFRP [16]	✓	✓	✓	∼
CFRP [21]	∼	✓	✓	∼
Haai + Remus [7], [8]	✓	✓	×	✓
Juniper [22]	×	✓	✓	∼
E-FRP [20]	✓	✓	✓	∼

Table 2.1: Candidate RP languages for MCUs against the requirements. ✓: met, ∼: partially met, ×: not met or not shown in the cited work.

As Table 2.1 shows, none of these languages meets all the requirements. However, some requirements, such as interoperability, can be met by building additional tooling. Juniper has no predictable memory footprint [22], so we leave it out. E-FRP has a similar design to EmFRP [11], but it is a prototype that generates C for the compiler of one specific MCU, the PIC16C66 [20]. EmFRP, in contrast, generates platform-independent ANSI C that can run on multiple MCUs, and offers more language features, such as modules, algebraic data types and type inference [16]. Therefore, we compare the following languages: EmFRP, CFRP and Haai + Remus.

2.2.1 EmFRP

EmFRP is a reactive language designed to work on small-scale embedded systems that cannot run a full-fledged operating system such as Linux [16]. EmFRP works as follows: the developer declares a series of nodes, which can be seen as signals. These nodes can refer to other nodes,

which defines the dependencies between nodes. These connected nodes form a directed graph. EmFRP does not allow recursion; instead, EmFRP allows cycles through the `@last` annotation, which allows referring to the previous value of any node [16]. Restricting recursion allows determining the memory footprint at compile time [16].

EmFRP's prototypical example is a thermostat, shown in Listing 2.6. We start by declaring the module name, the program's inputs, in this case the sensed temperature and the periodical pulse, and the outputs, in this case the heater state (on/off), followed by the used libraries. Afterwards, a single node is declared that periodically turns the heater on if the temperature is below 50 and off otherwise. Between pulses, the heater keeps its state [16].

```
module Thermostat
in
    temperature : Int,  # temp sensor
    pulse1min : Bool    # periodical pulse
out
    switching : Bool    # heater state
use
    Std                 # standard library

node init [ False ] switching =
    if pulse1min then
        temperature < 50
    else
        switching@last  # previous value
```

Listing 2.6: Thermostat controller in EmFRP [16]

EmFRP's example does give rise to the question of how peripherals are connected. EmFRP's compiler compiles the EmFRP program into two C files: on the one hand it generates FRP C code and on the other hand an I/O code skeleton, shown in Listing 2.7. The developer is required to fill in the I/O skeleton with the platform- and/or hardware-dependent code so the FRP code can interact with the hardware [16]. After the I/O code is filled in, the developer can use a C compiler targeting the platform to get the appropriate binary [16].

2.2.2 CFRP

Another RP language targeting embedded devices is CFRP [21]. CFRP was created as a pure FRP language that compiles to standalone C++ code. It was created to show that FRP is beneficial for developing software for small-scale embedded devices.

It was designed as follows: a CFRP program consists of a series of declarations, which denote signals and links to hardware (in C++), followed by a single Haskell-like expression that builds the reactive system [21]. The difference with EmFRP is that CFRP uses a single expression to construct a dependency graph, which is then used to compile to the C++ program. EmFRP uses nodes to construct the dependency graph. Additionally, CFRP allows anonymous functions, whereas EmFRP drops them. Moreover, CFRP has support for first-class signal

```c
#include "Thermostat.h"
void Input(int *temperature, int *pulse1min){
    /* Your code goes here ... */
}

void Output(int *switching){
    /* Your code goes here ... */
}

int main(){
    ActivateThermostat();
}
```

Listing 2.7: I/O Code Skeleton for Thermostat [16]

types, whereas EmFRP does not. These features do come with a cost, since EmFRP drops these features to keep its memory use predictable [21]. The caveat with CFRP is that the C++ code the program references has to be defined in separate C++ files [21]. This means that only the 'global' reactive system is defined in CFRP and the rest in C++.

An example can be seen in Listing 2.8, where the air conditioner is turned on or off depending on a discomfort index. The discomfort index is calculated from the temperature `tmp` and humidity `hmd`. CFRP uses 'base' types that resemble C++ types, but signals have an additional annotation `Signal`. Wrapping base types or tuples of base types into a `Signal` type makes the size of signals predictable at compile time [21].

```
-- Including C++ header files
%{
#include "sensors.h"
#include "ac_ctrl.h"
%}

-- Declaring identifiers connected to external devices
%input tmp :: Signal Float = sen_tmp;
%input hmd :: Signal Float = sen_hmd;
%import ac :: Signal Bool → Signal () = ctrl_ac;

-- Main expression
let di t h = t - 0.55 * (1 - 0.01 * h) * (t - 14.5) in
    ac (lift1 (\x → x > 24) (lift2 di tmp hmd))
```

Listing 2.8: Example program in CFRP [21]

2.2.3 Haai + Remus

Haai is a pure RP language in which programs are composed of reactors and deployments of reactors. We call Haai 'pure' because everything is reactive, even its primitives [3]. Additionally, Haai is an independent language, meaning it does not rely on a host language, the way REScala that relies on Scala [1]. Nowadays, Haai comes with a virtual machine (VM) called Remus, which primarily allows Haai to run on the bare metal [7], [11]. Remus was later implemented in Rust and with some modifications a prototype was created to run a Haai program on an M5StickC [8].

The benefit of Haai is that it provides platform independence through its compilation step from Haai to Remus bytecode. All that is required is a Remus bytecode interpreter for the target device, in the language of choice. Additionally, Remus is well suited for constrained devices as all the memory a well-behaved program requires is allocated up front, achieving a predictable memory footprint [7]. Moreover, the Rust implementation of Remus provides the necessary interfaces to interact with sensors and actuators [8], leaving interoperability as the only hurdle of this language. Fortunately, as we will see in Section 2.4, this can be solved by implementing Remus in a different low-level language that the chosen robotics framework does support.

Having discussed EmFRP, CFRP and Haai, we can conclude that they all meet most criteria, though not all. Unfortunately, EmFRP and CFRP are partially portable, as they have to be recompiled each time for their target platform. Additionally, CFRP does not give a bound in terms of memory usage. Haai, in contrast, is compiled once and then can be run anywhere with a Remus VM. Additionally, the access to the Haai language makes it practical to make necessary language extensions, should they be required. Therefore, this thesis will build on Haai and Remus.

In this thesis we focus on a system of MCUs. Take a robot, for example: a robot has sensors as input and has other peripherals that are then used to interact with the real world, depending on these inputs. In other words, the robot reacts to the sensors by instructing the motors what to do or where to move next.

To build this system of embedded devices, we could create some reactive programs in Haai, but this would leave the developer responsible for implementing all the communication between the devices as well as some central computing unit that coordinates all these devices and peripherals, most of the time requiring the developer to use a library that uses some kind of communication protocol under the hood (Bluetooth, UART, I2C, SPI, USB, etc...). A standardized toolkit that abstracts these complexities away would be easier, which is what we will be looking into next.

2.3 Robotics frameworks

In this section we will discuss robotics frameworks that are open source, provide a software development kit (SDK) to aid development and have a user base: Robot Operating System (ROS) [4], [23], [24], [25], Open Robot Control Software (OROCOS) [5] and Yet Another Robot Platform (YARP) [6]. We will analyse each framework, see how it works, if it runs on MCUs

and whether it abstracts away the communication between devices.

2.3.1 ROS

Originating from Stanford, Robot Operating System, or ROS, is in fact not an operating system but rather an SDK [24]. To clarify, when we speak of ROS, we refer to ROS 2, unless stated otherwise.

Architecture

ROS provides modules to aid in the development of robotics systems. The modules can be categorized as follows [24]:

- Middleware: the communication between components.
- Algorithms: libraries of commonly used algorithms.
- Developer tools: tools to configure, launch, inspect and debug ROS applications.

The middleware is essentially 'a structured communications layer above the host operating systems of a heterogenous compute cluster' [23]. A ROS cluster consists of several on-board devices and off-board devices, which can connect and disconnect at runtime. Consequently, the designers of ROS chose a peer-to-peer (p2p) system as having a central server would mean a lot more network saturation as messages would have to go to a central point and would then have to be forwarded to their respective recipients. In a p2p system, nodes communicate directly, without a central system, leading to less network saturation and shorter distances travelled [23].

The philosophy of ROS of using multiple standalone components within its system allows for developers to reuse or repurpose existing hardware libraries within ROS, but without actually depending on ROS. This also allows for robust testing of these libraries, without being dependent on ROS. The decoupling of these components also means that they can be used in other robotics systems where there is no inherent dependency and vice versa, giving ROS access to existing standalone components, such as drivers and vision algorithms [23].

Communication

A ROS application consists of nodes, which are processes that perform a computation. Multiple of these form a larger system that mimics the structure of a graph. Note that a node is not necessarily mapped to one device, as a device could run a single node or many. To communicate between these nodes, messages are sent. To facilitate communication, ROS provides three abstractions to send these messages: topics, services and actions [24].

Figure 2.2 shows a robotics example where some products are being transported down a conveyor belt. However, these products have to be classified by colour, which the sensors on the

'sensors' device can detect. The sensors can then send the detected colour to the operator computer, which selects the appropriate sorter to sort the product. Each sorter consists of an arm and a sensor. The sensor detects when the product is in the sorting range and sends a signal to the arm, which sorts it. Furthermore, the operator computer shows the detected colours on the screen and allows the user to manually operate a sorter if need be thus completing the graph shown in the figure.

Messages that these nodes send to each other are actually strictly typed data structures. These can consist of the ROS-provided primitive types (integer, floating point, boolean, etc...), arrays of primitive types as well as constants though they must be defined in a `.msg` file, an interface definition language (IDL). An example can be seen in Listing 2.9. Messages can also be composed of other messages or arrays of messages which can be nested arbitrarily deep. Do note that ROS automatically generates native types for the types used in the IDL, depending on the language where ROS is used. For instance, if the node is written in C++, ROS will (de)serialize to C++ types [23]. This also counts for nodes running on Python or C, just like JSON is automatically (de)serialized in other high-level programming languages. The ROS Client library that provides the API to create a ROS graph is called rcl, which is written in C. Depending on the language used, the API is built upon further resulting in rclcpp (C++), or rclpy (Python).

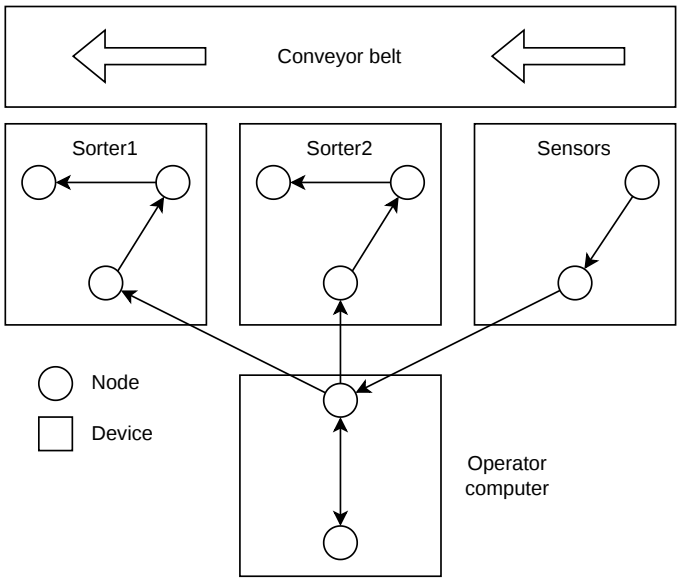


Figure 2.2: ROS node graph example

Within ROS, topics are the main communication abstraction, as actions are built on top of topics and services [26]. A topic is essentially a channel that allows nodes to publish or subscribe to the topic. This allows for a many-to-many relationship within the ROS node graph. An illustration of Figure 2.2 implemented using topics can be seen in Figure 2.3. Topics allow for unidirectional one-to-one, one-to-many, many-to-many or many-to-one relationships.

```
# ProductColour.msg
uint64 timestamp_ms
string colour
```

Listing 2.9: Message definition for a detected colour

A bidirectional relationship between two nodes can be created by making one of the nodes a subscriber to a topic and the other a publisher as well as the inverse on another topic as seen on the operator computer.

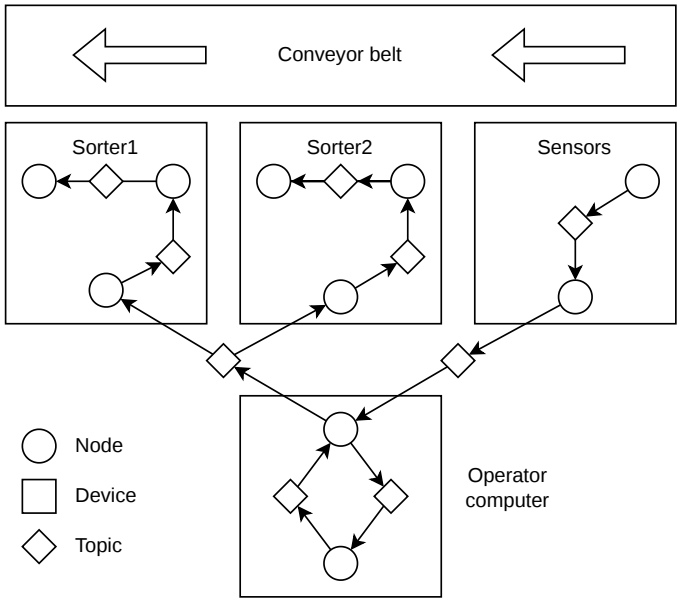


Figure 2.3: ROS node graph with topics

Whilst topics are the main communication abstraction that ROS uses, they are lacking for communication where a message sent expects a response. To solve this, ROS provides services, which work analogously to web services. A node places a request to a uniquely named service that sits on another node, which returns a response after processing the request, just like web services [23]. This is shown in Figure 2.4.

Like messages, the interface of services must be declared in an IDL. In this case it will be in a `.srv` file instead of a `.msg` file, though the same principle applies. The only difference is that the interface is split in two. Above the `---` is the interface of the request and below the interface of the response. An example can be seen in Listing 2.10, where we send a request to a node to get the number of items a specific sorter has sorted. The sorter is identifiable by `sorter_id` and the response is `count`.

In case both topics and services are inadequate and you need the best of both worlds, there are

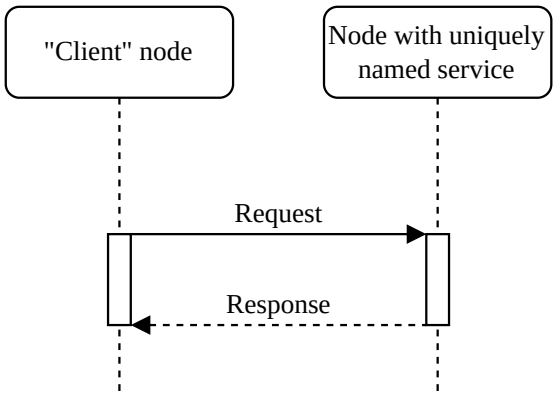


Figure 2.4: ROS service protocol

```
# SorterService.srv
uint32 sorter_id
---
uint32 count
```

Listing 2.10: Service definition for the count of a sorter

also actions. Actions are a combination of both topics and services and ideal for long-running tasks and work as follows: an action starts off like a service, a node sends a request to a uniquely named action, which in turn has to process the request. During the processing of the request the uniquely named action produces some feedback which gets published to a topic and is sent to its subscribers. After processing the request a response is sent back to the node that made the request. Unlike services or topics, actions allow for cancellation of the request if needed. A diagram showing the flow of actions can be seen in Figure 2.5.

Like the interface of messages and services, the interface of actions must be defined in an `.action` file. Like the interface of services, the interface of actions is also split up but three-way instead of two-way. The first segment declares the interface of the request. The second of the response and the third segment declares the interface of the messages published to the topic. An example can be seen in Listing 2.11. In this example the operator computer tells a specific sorter to sort a batch, whilst keeping track of the number of sorted items so far, and finally receives the total count of sorted items.

```
# SortBatch.action
uint32 sorter_id
uint32 batch_size
---
uint32 sorted_total
---
uint32 sorted_so_far
```

Listing 2.11: Action definition for sorting a batch of items

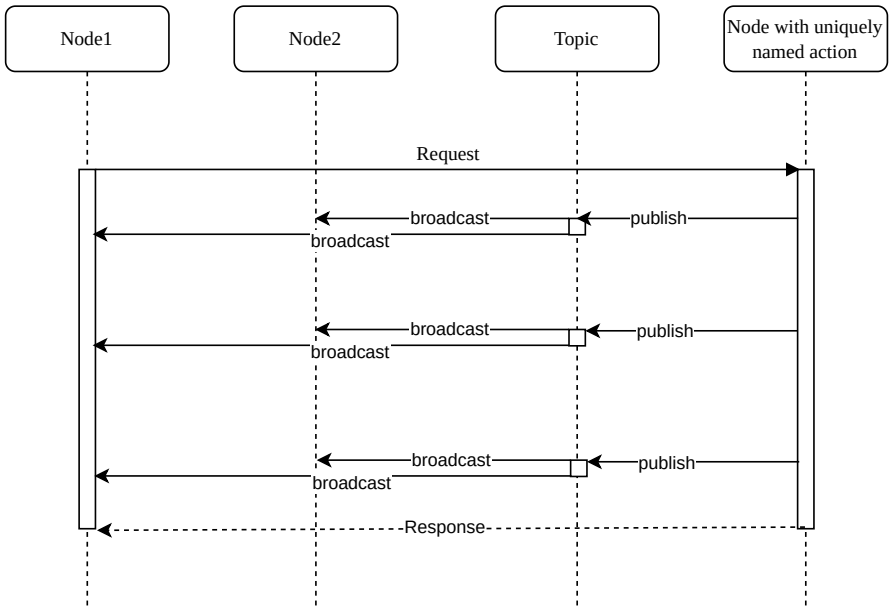


Figure 2.5: ROS actions protocol

Table 2.2 summarizes the three communication abstractions of ROS.

Abstraction	Interface	Pattern	Suited for
Topic	`.msg`	Pub-Sub	One-way communication between nodes
Service	`.srv`	Request-response	Short tasks that return a response
Action	`.action`	Request, feedback and response	Long-running tasks that report progress and can be cancelled

Table 2.2: Communication abstractions in ROS

History of ROS

ROS 1 is the predecessor of ROS 2 as the number probably already suggested, however they are vastly different. As ROS 1 started as a research platform and as it grew in popularity across robotics, its limitations became increasingly evident. 'Security, reliability in nontraditional environments, and support for large-scale embedded systems became essential' [24]. Since ROS 1 did not provide solutions for these concerns, developers were forced to create application-specific workarounds [24], which could not be reused. To solve these issues, ROS was rebuilt from the ground up and thus ROS 2 was born [24]. A summary of the comparison of features between the versions can be seen in Table 2.3. Without getting into too much of the technical changes, the most noteworthy changes would be the new underlying network

Category	ROS 1	ROS 2
Network transport	Bespoke protocol built on TCP/UDP	Existing standard (DDS), with abstraction supporting addition of others
Network architecture	Central name server (roscore)	Peer-to-peer discovery
Platform support	Linux	Linux, Windows, and macOS
Client libraries	Written independently in each language	Sharing a common underlying C library (rcl)
Node versus process	Single node per process	Multiple nodes per process
Threading model	Callback queues and handlers	Swappable executor
Node state management	None	Lifecycle nodes
Embedded systems	Minimal experimental support (rosserial)	Commercially supported implementation (micro-ROS)
Parameter access	Auxiliary protocol built on XMLRPC	Implemented using service calls
Parameter types	Type inferred when assigned	Type declared and enforced

Table 2.3: Summary of ROS 2 features compared with ROS 1 [24].

communication, the Data Distribution Service (DDS), which is also used in critical infrastructure [24], peer-to-peer discovery and the support for embedded systems.

The change to DDS as its network communication solved many of the problems, as DDS enables ROS 2 'to obtain best-in-class security, embedded and real-time support, multirobot communication, and operations in nonideal networking environments' [24]. DDS provides peer-to-peer discovery, which is one of ROS 2's core defining features, as DDS is the underlying medium that the communication abstractions depend on.

MCU support

Unlike ROS 1, which only had minimal experimental support through rosserial, ROS 2 supports embedded systems [24]. It does so because sensors and actuators of a robot are often driven by MCUs, which are constrained in memory and are unable to run a ROS system. Yet these MCUs are still expected to communicate with a CPU running ROS [24]. ROS 2 supports MCUs by providing a dedicated SDK called micro-ROS [25] for resource-constrained devices (such as MCUs). The SDK provides all major features that the ROS 2 SDK provides by reusing as many packages as possible and providing them in C by extending the existing ROS client library (rcl), which is written in C, with the rclc packages. Like rclcpp does for C++, together they form a complete client library for C [25].

To be able to provide all these major features, micro-ROS replaces DDS with XRCE-DDS, which is a variant of DDS for extremely resource-constrained environments [27]. An agent running on a CPU bridges between XRCE-DDS and DDS, allowing MCUs to communicate with nodes running ROS [25].

Unfortunately, micro-ROS cannot be used on every single microcontroller. As ROS 2 was initially developed for POSIX-based operating systems, some clock and timing functions remain of the essence. This means that the microcontroller has to either be able to run an RTOS providing these functions or have abstractions providing them. Currently, micro-ROS supports some Real-Time Operating Systems, such as FreeRTOS, Zephyr and NuttX, as well as some boards such as Espressif ESP32, Raspberry Pi Pico, some Arduino boards and some STM boards [25]. There is also experimental support for bare-metal usage, other RTOSes and other boards. Boards contributed by the community are not guaranteed to be supported, whereas officially supported boards, such as the Raspberry Pi Pico, are tested and receive long-term support [28].

Whilst micro-ROS tries to provide as many of the features that ROS 2 provides, there are some considerations that have been made, as the environment has resource constraints, and thus cannot provide all features [25]:

- The messages interchanged between two nodes, including at least one micro-ROS node must be a fixed size. This is to avoid dynamic memory allocations, which is heavily discouraged to do on a microcontroller. This is because it is a best practice to allocate all the memory you need upon initialization, as running out of memory during runtime could yield undefined behaviour.
- To deal with different mechanisms and scheduling APIs of the underlying RTOS, rclc provides a flexible Executor on the level of individual callbacks.
- ROS 2 provides a launch system, which starts and configures many nodes with a single command [29]. Micro-ROS does not have this due to different underlying RTOS mechanisms and leaves it up to the developer.
- Standard ROS 2 allows for dynamic loading of nodes into running processes. Since the program has to be flashed onto the microcontroller, the node composition of the micro-ROS program has to be known ahead of time, and thus at compile-time.
- The XRCE-DDS standard defines no security and thus relies on the mechanisms provided by the underlying transport medium. If the medium (e.g., UART) has no security, then the connection is not encrypted. Though for wired connections, this is generally considered acceptable.

To summarize, ROS is an SDK which provides modules to aid robotics development. Most notable is its communication layer, which allows for peer-to-peer communication. ROS also provides an SDK for development on microcontrollers, though not all microcontrollers. ROS thus supports MCUs, provides common libraries and handles the communication between devices for us.

2.3.2 OROCOS

Open RObot COntrol Software (OROCOS) started at the Katholieke Universiteit Leuven (KUL) as a project that ‘aims at becoming a general-purpose and open robot control software package’ [5].

Architecture

OROCOS is a free software framework aimed at real-time control of robots and machine tools [30]. Its toolchain is used to build real-time software components and also takes care of the communication and execution of and between these components [31]. A component can be seen as a unit of functionality that runs one or more programs on a thread [31]. OROCOS consists of the following parts:

- The OROCOS toolchain

  The OROCOS toolchain is composed of the Real-Time Toolkit (RTT), the OROCOS Component Library (OCL) and finally code generation tools for C++ data types and ROS messages. The RTT can be seen as the middleware between the application and the operating system (OS). It abstracts the OS, runs the components, and handles communication between components. The OCL provides prebuilt components that are commonly used in an OROCOS system, logging components for example [31].

- Standalone libraries

  These are libraries that were created to be independent of the OROCOS toolchain. This means you are able to use them outside of an OROCOS project. The libraries are the Kinematics and Dynamics Library (KDL) and the Bayesian Filtering Library (BFL) [30]. KDL is used to compute positions and motions of joints and links of robots [32]. BFL is used to estimate the true state of a robot from noisy sensor data [33].

- Integration and tooling

  This part is composed of integrations to interoperate with another robotics framework, interestingly, ROS [34], [35], which is discussed in Section 2.3.1. OROCOS provides `rtt_ros2_integration` to interoperate OROCOS components with ROS nodes [35]. Additionally, this part consists of a tool to generate motions of robots from constraints (iTaSC) [36] and a tool to coordinate robots with state machines (rFSM) [37].

Communication

OROCOS offers abstractions to communicate between components. The abstractions can be classified under the following two categories: data flow ports and operations.

Data flow ports can be seen as a many-to-many communication. Conceptually it can be seen as publish-subscribe. Each component declares input ports (`InputPort<T>`) and output ports (`OutputPort<T>`), where `T` is the data type of the payload that is transmitted between ports; this can be any C++ data type, struct or class, for example `double`. Each `OutputPort<double>` can be connected to multiple `InputPort<double>` of a different component, and multiple `OutputPort<double>` can be connected to a single `InputPort<double>`. The connection itself can be created outside the component through a script or an XML file. The usual way these connections are created is by using the deployer, an OCL component that runs the script or XML file containing the connection declarations. However, you can also create the connections in your own C++ program [31].

Operations are essentially methods that are provided by components that can be called by other components through an `OperationCaller`, which is a helper object for calling operations [31]. Conceptually, this is similar to Remote Method Invocation (RMI). The component that wants to provide the operation registers the method or function as an operation in its interface. This makes the operation available to other components, and can be done by using `addOperation` in the component's constructor. It registers the operation under a name, with the associated function and component, and declares where the function is meant to run: either in the component's own thread (`OwnThread`) or in the caller's thread (`ClientThread`), which often requires using a mutex to be thread safe. This can be seen in Listing 2.12. Other components can in turn fetch the operation from the component and save it to an `OperationCaller` helper with the same signature as the registered method. Once the component needs to invoke the operation, this can be done synchronously by calling the method like you would call a function, or asynchronously by calling `send()` on the `OperationCaller`, which returns a `SendHandle`. The result can later be collected by using `collect()` or `collectIfDone()` on the handle, as seen in Listing 2.13 [31].

```
class Sorter : public RTT::TaskContext {
public:
    uint32_t sorted_items = 0;

    void operate(){
        // logic to sort
        this->sorted_items++;
    }

    uint32_t getSortedCount(){ return this->sorted_items; }

    Sorter(std::string name): TaskContext(name) {
        this->addOperation("operate", &Sorter::operate, this, RTT::OwnThread);
        this->addOperation(
            "getSortedCount",
            &Sorter::getSortedCount,
            this, RTT::OwnThread);
    }
};
```

Listing 2.12: Registering methods as operations in OROCOS

Under these abstractions OROCOS makes use of transports, which are a layer that carries ports and operations from one process to another. The first transport is CORBA, which is used between processes and across the network. This allows components to live in separate processes or even devices and still be able to communicate with each other. It works for both ports and operations [31].

The second transport is called MQueue. MQueue allows for real-time communication between processes on the same device, as it uses POSIX message queueing available on Linux systems. This transport is limited to ports [31]. Additionally, OROCOS can make use of most of ROS's communication abstractions through its ROS integration library [34], [35].

```
TaskContext* sorter = getPeer("sorter");
OperationCaller<uint32_t(void)> getSortedCount =
sorter->getOperation("getSortedCount");

uint32_t count = getSortedCount(); // synchronous call

SendHandle<uint32_t(void)> handle = getSortedCount.send(); // asynchronous call

while (handle.collectIfDone(count) == SendNotReady){
    // do work whilst getSortedCount runs asynchronously
}

handle.collect(count); // Alternative way to collect async result but blocking
```

Listing 2.13: Fetching an operation and calling or sending it in OROCOS

History of OROCOS

Before OROCOS, most of the robotics software was closed source and did not allow developers to go into the deepest layers of hardware control [5], [30]. This led to disappointing experiences for Herman Bruyninckx, who came up with the idea for OROCOS [30].

Originally, the plan was to split OROCOS into three parts: supporting modules, robotics modules and components [5]. Supporting modules would come from neighbouring projects whilst the robotics modules would be a collection of commonly used algorithms [5]. Today the robotics modules are the KDL and BFL libraries [30]. Components were originally CORBA objects which were built from the other two types of modules. Developers could use these objects to create their robot control software [5]. Nowadays components are built using `TaskContext` from the RTT and CORBA is used as a means of transport [31].

As of November 2026, the repositories containing the source code of OROCOS show that only KDL is actively maintained and the others have had their last maintenance during 2020–2022 [38], [39], [40], [41]. OROCOS wants to provide the best open source software within its niche. It provides KDL and BFL as standalone libraries and allows other frameworks to use them [30]. This leads to OROCOS often not adopting extensions and features into the project but instead delegating them to neighbouring projects such as ROS, which is in line with OROCOS providing the integration library with ROS [30], [34], [35].

MCU support

Originally, OROCOS was designed so that it could even run without an OS [42]. However, the implementation only ever targeted Linux and variants of Linux [31], [42]. In principle, it should be possible to port it as long as you can use an RTOS with full C++ support, as you need time reading, mutexes, semaphores and condition variables [31]. Such a port, to our knowledge, does not exist. Although the manual mentions that OROCOS can be used on embedded devices with less than 1 MB of memory [31], this refers to the smallest deployment on a Linux target.

Additionally, the transport layers assume a full OS, as they rely on POSIX message queues, an ORB and a naming service, and OROCOS does not provide a lightweight alternative for microcontrollers [31]. This means that OROCOS is inadequate for building a distributed system of microcontrollers, as it has yet to prove that it can run on microcontrollers. However, it could be used as a supporting tool within a different robotics framework, just as KDL is used within ROS [30].

2.3.3 YARP

Yet Another Robot Platform or YARP, developed at the LIRA-Lab and MIT, was created for the application of humanoid robotics, with a focus on applications that are real-time, computation-intensive and involve interfacing with diverse and changing hardware [6]. YARP categorizes itself as a middleware for robotics [43]. It is a set of libraries, protocols, and tools to keep modules and devices decoupled [44]. The goal of YARP is 'to minimize the effort devoted to infrastructure-level software development by facilitating code reuse, modularity and so maximize research-level development and collaboration' [6].

Architecture

The architecture of YARP can be split into three parts: the OS, signal and device libraries [44].

- OS library (`YARP_os`)

  Just as the RTT is the core of OROCOS, `YARP_os` is the core of YARP. It provides an OS layer abstraction so YARP can remain OS independent [44]. It also contains the communication abstractions, which are described in the Communication part below.

- Signal library (`YARP_sig`)

  Since humanoid robots often require vision and use cameras to see, or microphones to hear, libraries to process these signals are mandatory for YARP [6]. It supports images, audio, vector and matrix data as well as 3D point clouds (points that describe the surface of an object) and measurements from lasers. Interestingly, most of these data types implement YARP's `Portable` interface, meaning they are serializable out of the box [45].

- Device library (`YARP_dev`)

  The device library offers interfaces for common devices used in robotics as well as drivers implementing those interfaces for concrete devices [44], [46]. Moreover, since some devices also rely on network communication, YARP wants a scalable way to communicate across devices. To solve this, the library offers network wrappers so devices have a common interface with which they can communicate [46], [47]. This makes communication device- and location-agnostic.

Communication

YARP processes communicate with each other by making use of the Observer pattern and port objects. Port objects are active objects that manage multiple connections. Each connection has a state that can be manipulated from external commands, which manage the connection or obtain state information from it. Ports can either be declared as an input, or as an output. An input port can receive from many connections over different protocols. An output port can send to many receivers [6]. All ports get a unique readable name and are kept track of in the name server. The name server returns the address of the associated port when looked up by name. After this, communication goes straight to that address [6], [48].

Underneath port objects, YARP uses low-level protocols to communicate, called carriers. YARP provides the following carriers [6], [48]:

- TCP, a reliable protocol. TCP guarantees delivery.
- UDP, a faster protocol than TCP, but without a delivery guarantee.
- Multicast, used for one-to-many connections. This is efficient when wanting to share the same information across many processes.
- Shared memory. This is used for local connections when possible.
- QNet, a fast and synchronous protocol, used under the QNX real-time OS.

YARP provides serialization for all simple data types (i.e., not containing pointers) out of the box. More complex data types using pointers must first have their pointers recursively dereferenced and require appropriate (de)serialization functions [6].

YARP also provides Remote Procedure Call (RPC) ports to communicate between processes, which are an additional layer on top of ports. The difference lies in how they handle sends and receives. Regular ports are asynchronous, meaning that the sender does not wait for the receiver to process anything. RPC ports are synchronous because they actually wait for the receiver to send a response [49].

Like OROCOS, YARP also provides interoperability with ROS 2, though it does not allow communication between ports and ROS 2 nodes like OROCOS does [35]. Instead, YARP provides wrappers for devices in its device library to interoperate with ROS 2 called `yarp-devices-ros2` [47], [50]. This allows these devices to become ROS 2 nodes with fixed topics or services.

History of YARP

Humanoid robots need a lot of computing power. The only realistic way to achieve this is by having a cluster of computers. YARP's design revolves around how the processes on these computers can communicate with each other as easily as possible. Over the years the developers of YARP have learned the following lessons that make YARP what it is today [6]:

- A robot control system consists of multiple processes. This means that the developer must be able to design the system as a set of processes running on a set of computers. This maximizes the amount of time spent researching instead of optimizing code. Consequently, writing and running processes has to be as easy as possible.
- Whenever a single process has a single responsibility, it can be reused on any computer without code changes and can be moved during run time to spread the computational load or to recover from a hardware failure.
- Since the actual performance of a robot controller depends on the timing of various signals, existing processes should feel minimal interference when another process is created within the system.
- Since some processes could depend on other processes, it would be ideal to minimize these dependencies to prevent expensive chains or process restarts.
- YARP is meant to be used alongside external libraries that themselves do not necessarily depend on YARP. YARP does not try to be in control of the project and instead aims to support the project by allowing the developer to only use parts of YARP it needs.
- YARP is designed to be OS independent. Sometimes some tools are better in one OS rather than another. YARP uses an open source library that provides a concurrent framework across a wide variety of operating systems [6].

During the ROS 1 era, YARP was able to communicate directly with ROS 1 through nodes and the publish-subscribe architecture [43]. However, since YARP 4.0, it has dropped support for ROS 1 in favour of the device wrappers that support ROS 2 [51], [52], which are still being maintained [50].

MCU support

YARP officially supports Ubuntu 22.04, Windows 10 and macOS 12/13 [45], and it has also been used on iOS, Solaris and Android as well [44]. These are all OSes, meaning YARP has to run on a device that at the very minimum runs one of the mentioned OSes. This implies that like OROCOS it is short of supporting microcontrollers, as they often do not have the resources to run such a heavy OS. However, YARP can interact with microcontrollers as it has with iCub, a robot YARP was used for [53]. iCub allows interoperability with MCUs by providing drivers from which the computer can steer the MCUs [6], [44]. Despite YARP being able to instruct MCUs through these drivers, MCUs require firmware outside of YARP [54]. This means that YARP is also inadequate for the microcontrollers requirement.

2.4 Revisiting the Problem Statement

At the end of Section 2.2, a reactive language that targets MCUs was chosen: Haai with the support of Remus. This came with the caveat that a Remus implementation would have to be made if the language of the chosen robotics framework differs from existing Remus

implementations. Afterwards, three robotics frameworks were discussed in Section 2.3, namely ROS, OROCOS and YARP. Since OROCOS and YARP are inadequate for microcontrollers, ROS is the only remaining choice, as it provides support for microcontrollers through micro-ROS.

Reusing the product sum example in Figure 1.1, suppose this program is distributed across three devices, two reading the inputs `x` and `y` respectively. These devices communicate their values to a third device that performs the product and sum. Using RP alone, three separate programs would have to be made to be distributed and deployed on these different devices. Additionally, the communication layer would have to be written, which also depends on the medium used. This is called *distributed reactivity* [1] and leads to reactivity within the devices, but the glue between the devices hides the dependencies in the communication code, making the data flow no longer explicit.

ROS takes care of the communication code and allows programs to be structured in a node system. For instance, a signal can be rewritten as a ROS node and the dependencies can be written as topics. Each signal that depends on another signal subscribes to the topics of the signals it depends on. As an example, the Haai program from Listing 1.2 can be rewritten into a ROS system, split by nodes and topics as shown in Figure 2.6. The two input devices publish their `x` and `y` inputs to topics, to which the product sum device is subscribed. The product sum device computes the product sum and further propagates it in the system. The downside is that this would have to be created using callbacks, which brings back the problems discussed in Section 2.1. This has not changed with ROS 2, although there have been efforts to bring RP to ROS 2 through RxROS [10].

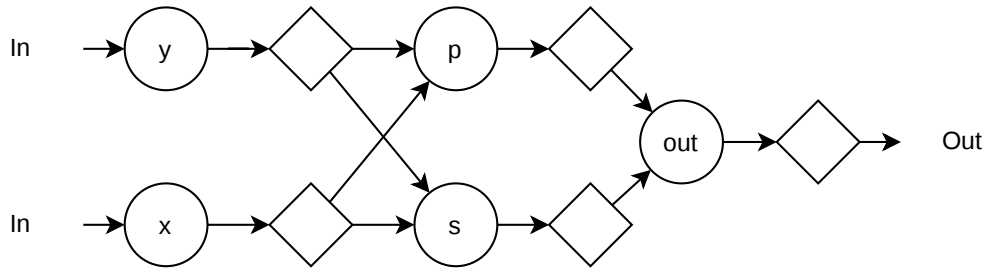


Figure 2.6: Product sum in a ROS node system

Despite RxROS allowing RP in a ROS system, the programs would still have to be written independently of each other [10], which is a problem this thesis attempts to solve by creating a distributed reactive program from a single source. Additionally, RxROS is not supported in micro-ROS due to RxROS being implemented in C++/Python whilst micro-ROS only provides support in C, which limits its usability [10], [25].

Since micro-ROS only supports C, and existing Remus implementations are in Rust or Racket [8], [55], the first contribution of this thesis is a C port of Remus (Chapter 3) so the bytecode of Haai can be run on MCUs. This solves running micro-ROS nodes. However, since ROS 2 is not restricted to C, there is more freedom in deciding how the ROS 2 parts of the program are run. A solution could be to reuse the C implementation of Remus, resulting in a single implementation for all devices. Alternatively, the existing Racket implementation could

be used through a foreign function interface (FFI) to rcl.

Despite these solutions, separate reactive programs would still have to be created, deployed and glued together by hand. Tierless programming addresses this.

2.5 Tierless Programming

Tierless (or multitier) programming is the practice of writing code for a distributed application in a single program, whose tiers can each run on a different machine on a network [56]. Once the tierless program is written, it is automatically split up by the compilation step, which decides the split based on the annotations, static analysis or types. The communication between the tiers is generated along with it [56].

In a traditional distributed system, such as a web environment, a developer would have to not only develop client-side code as well as server-side code, but also code gluing these tiers together [56]. This gives the illusion of decoupled tiers while in actuality they work closely together and depend on each other. To avoid writing extra glue code or huge API boilerplates, these tiers could be unified into a single source.

Writing a tierless program in a single source saves the developer from writing a lot of boilerplate code or glue code, allowing the developer to solely focus on the high-level interactions. Additionally, it saves the developer from having to design inter-tier APIs. For instance, this would abstract away the topics and the messages when compiling a Haai program to a ROS program. How the tiers are connected becomes a compilation step rather than a development step [56].

In traditional distributed programming, the boundaries between hosts and functionalities may not always coincide. In other words, a single functionality may be distributed across many places and many functionalities could be isolated to a single place. Whenever functionalities are split across different devices, changing them could become cumbersome as the same functionality is implemented multiple times. Using tierless programming would allow the developer to implement it once and distribute it where it is needed [56].

Tierless languages differ in several ways, three of which are relevant for this thesis. The first is placement. Where each part of the code runs can either be declared explicitly or left for the compiler to decide [56]. Second is communication. Communication is either built into the language or provided by its standard library. These mechanisms range from RPC, message passing and publish–subscribe to reactive programming and shared state, to name a few [56]. This thesis uses publish–subscribe, as that is how topics in ROS work. Lastly, the topology provided by the tierless language differs. Most only provide a client–server topology, although exceptions exist, such as ScalaLoci [9], [56].

To give an example of how such a tierless program is written, consider the ScalaLoci program in Listing 2.14. The `@multitier` annotation declares the program as a multitiered program, meaning all the code for all peers is inside this object. The `@peer` annotations declare a peer of type `Server` and `Client` respectively. The tie inside these peers declares how peers are

connected to each other. The `Server` typed peer says that a server is connected to multiple clients through `Multiple[Client]`. The `Client` typed peer in turn says that it is connected to a single server through `Single[Server]`. The example continues by declaring a `message` stream placed on the client peers with `on[Client]`. The server in turn declares a `publicMessage`, placed with `on[Server]`, that aggregates the event occurrences of all the connected peers through `asLocalFromAllSeq` [9]. This is where the inter-tier communication happens. `map` simply returns the message and drops the sender. The body of the `main` function makes each client fetch the server's `publicMessage` stream using `asLocal`, and prints every message with `observe println`. The loop in turn reads input and fires it into the `message` event with `fire`.

```
@multitier object Chat {
    @peer type Server <: { type Tie <: Multiple[Client] }
    @peer type Client <: { type Tie <: Single[Server] }

    val message = on[Client] { Evt[String]() }

    val publicMessage = on[Server] {
        message.asLocalFromAllSeq map { case (_, message) => message }
    }

    def main() = on[Client] {
        publicMessage.asLocal observe println
        for (line ← io.Source.stdin.getLines)
           message.fire(line)
    }
}
```

Listing 2.14: Chat example from the ScalaLoci website [57]

Without tierless programming this program would have to be split up into two parts: the server-side code and the client-side code. Additionally, the communication part would have to be programmed explicitly. In this tierless programming example the server- and client-side code is written in a single source and the communication code is generated by the compiler.

Tierless RP exists, for instance in Gavial, ScalaLoci and XFRP [1]. However, most tierless languages target the traditional client–server architecture that dominates the web, where tiers have similar processing capabilities [56]. A computer running an OS and an MCU do not have the same capabilities, as an MCU is often constrained in its resources. Bridging this gap is the subject of the following chapters.

Chapter 3

μ-Remus

3.1 Haai & Remus

3.2 ROS & μ-ROS

3.3 Mapping RP to ROS & μ-ROS

3.4 μ-Remus: A C implementation

Chapter 4

Programming Robots with Reactors

4.1 Linking ROS and μ-ROS

Chapter 5

Tierless Haai

5.1 Haai to CHaai

Chapter 6

Evaluation

6.1 Evaluating Haai distribution

6.2 Evaluating Haai to CHaai

6.3 Evaluating CHaai on μ-Remus

Chapter 7

Conclusion & Future Work

Bibliography

[1] S. Van den Vonder, B. Oeyen, G. Salvaneschi, W. De Meuter and J. Nicolay, 'A survey on reactive programming and reactive streams,' *ACM Computing Surveys*, Nov. 2024, Manuscript submitted to ACM.

[2] E. Bainomugisha, A. Lombide Carreton, T. Van Cutsem, S. Mostinckx and W. De Meuter, 'A survey on reactive programming,' *ACM Comput. Surv.*, vol. 45, no. 4, Aug. 2013, ISSN: 0360-0300. DOI: 10.1145/2501654.2501666 [Online]. Available: https://doi.org/10.1145/2501654.2501666

[3] B. Oeyen, J. De Koster and W. De Meuter, 'Reactive programming without functions,' *The Art, Science, and Engineering of Programming*, vol. 8, no. 3, 11:1–11:30, 2024. DOI: 10.22152/programming-journal.org/2024/8/11

[4] A. A. Allaban, D. Bonnie, E. Knapp, P. Gokhale and T. Moulard, 'Developing production-grade applications with ros 2,' in *Robot Operating System (ROS): The Complete Reference (Volume 6)*, ser. Studies in Computational Intelligence, A. Koubaa, Ed., vol. 962, Cham, Switzerland: Springer Nature Switzerland AG, 2021, pp. 3–52, ISBN: 978-3-030-75471-6. DOI: 10.1007/978-3-030-75472-3_1

[5] H. Bruyninckx, 'Open robot control software: The OROCOS project,' in *Proceedings of the 2001 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 3, Seoul, South Korea: IEEE, 2001, pp. 2523–2528, ISBN: 0-7803-6475-9. DOI: 10.1109/ROBOT.2001.933002

[6] G. Metta, P. Fitzpatrick and L. Natale, 'YARP: Yet another robot platform,' *International Journal of Advanced Robotic Systems*, vol. 3, no. 1, pp. 43–48, 2006. DOI: 10.5772/5761

[7] B. Oeyen, J. Nicolay and W. De Meuter, 'A virtual machine for higher-order reactors,' in *Programming Companion 2024: Proceedings of the 8th International Conference on the Art, Science, and Engineering of Programming*, E. Soderberg and L. Church, Eds., New York, NY, USA: Association for Computing Machinery, 2024, pp. 52–56. DOI: 10.1145/3660829.3660840

[8] C. Descheemaeker, 'Compilation-based execution and optimisation of reactive programs on the bare metal: A vm approach,' Master's thesis, Vrije Universiteit Brussel, Brussels, Belgium, 2023.

[9] P. Weisenburger, M. Köhler and G. Salvaneschi, 'Distributed system development with ScalaLoci,' *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, 129:1–129:30, Oct. 2018. DOI: 10.1145/3276499

[10] H. Larsen, G. van der Hoorn and A. Wąsowski, 'Reactive programming of robots with rxros,' in *Robot Operating System (ROS): The Complete Reference (Volume 6)*, A. Koubaa, Ed. Cham: Springer International Publishing, 2021, pp. 55–83, ISBN: 978-3-030-75472-3. DOI: 10.1007/978-3-030-75472-3_2 [Online]. Available: https://doi.org/10.1007/978-3-030-75472-3_2

[11] B. Oeyen, J. De Koster and W. De Meuter, 'Reactive programming on the bare metal: A formal model for a low-level reactive virtual machine,' Vrije Universiteit Brussel, Brussels, Belgium, Tech. Rep. vub-tr-soft-22-15, 2022.

[12] J. Gubbi, R. Buyya, S. Marusic and M. Palaniswami, 'Internet of things (iot): A vision, architectural elements, and future directions,' *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, 2013. DOI: 10.1016/j.future.2013.01.010

[13] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, 'Edge computing: Vision and challenges,' *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198

[14] J. Liu et al., 'Space-efficient trec for enabling deep learning on microcontrollers,' in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '23, New York, NY, USA: Association for Computing Machinery, Mar. 2023, pp. 644–659, ISBN: 9781450399180. DOI: 10.1145/3582016.3582062

[15] A. Sharma, S. Sharma, S. R. Tanksalkar, S. Torres-Arias and A. Machiry, 'Rust for embedded systems: Current state and open problems,' in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '24, New York, NY, USA: Association for Computing Machinery, Oct. 2024. DOI: 10.1145/3658644.3690275

[16] K. Sawada and T. Watanabe, 'Emfrp: A functional reactive programming language for small-scale embedded systems,' in *Proceedings of the 15th International Conference on Modularity Companion*, ser. MODULARITY '16 Companion, New York, NY, USA: Association for Computing Machinery, Mar. 2016, pp. 36–44, ISBN: 9781450340335. DOI: 10.1145/2892664.2892670

[17] A. Sarkar and M. Sheeran, 'Hailstorm: A statically-typed, purely functional language for IoT applications,' in *Proceedings of the 22nd International Symposium on Principles and Practice of Declarative Programming*, ser. PPDP '20, New York, NY, USA: Association for Computing Machinery, Sep. 2020, 12:1–12:16, ISBN: 9781450388214. DOI: 10.1145/3414080.3414092

[18] A. Sterz et al., 'ReactiFi: Reactive programming of Wi-Fi firmware on mobile devices,' *The Art, Science, and Engineering of Programming*, vol. 5, no. 2, 2021, Art. no. 4. DOI: 10.22152/programming-journal.org/2021/5/4

[19] Z. Wan, W. Taha and P. Hudak, 'Real-time FRP,' in *Proceedings of the Sixth ACM SIGPLAN International Conference on Functional Programming*, B. C. Pierce, Ed., ser. ICFP '01, New York, NY, USA: Association for Computing Machinery, Sep. 2001, pp. 146–156. DOI: 10.1145/507635.507654

[20] Z. Wan, W. Taha and P. Hudak, 'Event-driven FRP,' in *Practical Aspects of Declarative Languages (PADL 2002)*, S. Krishnamurthi and C. R. Ramakrishnan, Eds., ser. Lecture Notes in Computer Science, vol. 2257, Berlin, Heidelberg: Springer, 2002, pp. 155–172. DOI: 10.1007/3-540-45587-6_11

[21] K. Suzuki, K. Nagayama, K. Sawada and T. Watanabe, 'CFRP: A functional reactive programming language for small-scale embedded systems,' in *Proceedings of the 6th Workshop on Computing: Theory and Practice (WCTP2016)*, S.-y. Nishizaki, M. Numao, J. D. L. Caro and M. T. C. Suarez, Eds., Singapore: World Scientific, 2017, pp. 1–13. DOI: 10.1142/9789813234079_0001

[22] C. Helbling and S. Z. Guyer, 'Juniper: A functional reactive programming language for the Arduino,' in *Proceedings of the 4th International Workshop on Functional Art, Music, Modelling, and Design*, D. Janin and M. Sperber, Eds., ser. FARM 2016, New York, NY, USA: Association for Computing Machinery, Sep. 2016, pp. 8–16. DOI: 10.1145/2975980.2975982

[23] M. Quigley et al., 'Ros: An open-source robot operating system,' in *IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open Source Software*, vol. 3, 2009, p. 5.

[24] S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall, 'Robot operating system 2: Design, architecture, and uses in the wild,' *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074 [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.abm6074

[25] K. Belsare et al., 'Micro-ros,' in *Robot Operating System (ROS): The Complete Reference (Volume 7)*, ser. Studies in Computational Intelligence, A. Koubaa, Ed., vol. 1051, Cham: Springer Nature Switzerland AG, 2023, pp. 3–55, ISBN: 978-3-031-09061-5. DOI: 10.1007/978-3-031-09062-2_2

[26] 'Understanding actions,' ROS 2 Documentation (Kilted), Accessed: 2nd Nov. 2026. [Online]. Available: https://docs.ros.org/en/kilted/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html

[27] Object Management Group, 'DDS for eXtremely Resource Constrained Environments (DDS-XRCE),' Object Management Group, OMG Formal Specification formal/2020-02-01, version 1.0, Feb. 2020. [Online]. Available: https://www.omg.org/spec/DDS-XRCE

[28] 'Supported hardware,' micro-ROS, Accessed: 3rd Nov. 2026. [Online]. Available: https://micro.vulcanexus.org/docs/overview/hardware/

[29] 'Launch,' ROS 2 Documentation (Kilted), Accessed: 3rd Nov. 2026. [Online]. Available: https://docs.ros.org/en/kilted/Concepts/Basic/About-Launch.html

[30] 'Orocos project history,' The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://orocos.org/content/history.html

[31] 'The Orocos component builder's manual.' version 2.6.0, The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://orocos.org/stable/documentation/rtt/v2.x/doc-xml/orocos-components-manual.html

[32] 'Kinematics and dynamics library (KDL),' The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://www.orocos.org/kdl.html

[33] 'The bayesian filtering library (BFL),' The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://www.orocos.org/bfl.html

[34] 'Orocos RTT / ROS integration packages (rtt_ros_integration),' The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://github.com/orocos/rtt_ros_integration

[35] 'Orocos RTT / ROS 2 integration packages (rtt_ros2_integration),' The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://github.com/orocos/rtt_ros2_integration

[36] 'iTaSC wiki,' The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://www.orocos.org/itasc.html

[37] 'rFSM statecharts,' The Orocos Project, Accessed: 5th Nov. 2026. [Online]. Available: https://github.com/orocos/rFSM

[38] 'Orocos kinematics and dynamics C++ library (KDL),' The Orocos Project, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/orocos/orocos_kinematics_dynamics

[39] 'Orocos real-time toolkit (RTT),' The Orocos Project, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/orocos-toolchain/rtt

[40] 'Orocos component library (OCL),' The Orocos Project, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/orocos-toolchain/ocl

[41] 'The orocos bayesian filtering library (BFL),' The Orocos Project, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/orocos/orocos-bayesian-filtering

[42] H. Bruyninckx, P. Soetens and B. Koninckx, 'The real-time motion control core of the Orocos project,' in *Proceedings of the 2003 IEEE International Conference on Robotics and Automation (ICRA)*, Taipei, Taiwan: IEEE, 2003, pp. 2766–2771. DOI: 10.1109/ROBOT.2003.1242011

[43] P. Fitzpatrick, E. Ceseracciu, D. E. Domenichelli, A. Paikan, G. Metta and L. Natale, 'A middle way for robotics middleware,' *Journal of Software Engineering for Robotics*, vol. 5, no. 2, pp. 42–49, 2014. DOI: 10.6092/JOSER_2014_05_02_p42

[44] 'What exactly is YARP?' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp/blob/master/doc/000_general/what_is_yarp.dox

[45] 'YARP – yet another robot platform,' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp

[46] 'Getting started with YARP devices,' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp/blob/master/doc/module_devices/1_note_devices.dox

[47] 'Network wrapper server and network wrapper client architecture,' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp/blob/master/doc/module_devices/5_nws_and_nwc_architecture.dox

[48] 'Definition of YARP terms,' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp/blob/master/doc/000_general/yarp-terms.dox

[49] 'Specialized RPC ports,' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp/blob/master/doc/010_ports/rpc_ports.dox

[50] 'YARP devices for ROS 2 (yarp-devices-ros2),' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp-devices-ros2

[51] 'YARP 4.0.0 release notes,' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp/blob/master/doc/release/v4_0_0.MD

[52] 'YARP devices for ROS 1 (yarp-devices-ros),' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/yarp-devices-ros

[53] M. Randazzo, A. Ruzzenenti and L. Natale, 'YARP-ROS inter-operation in a 2D navigation task,' *Frontiers in Robotics and AI*, vol. 5, p. 5, 2018. DOI: 10.3389/frobt.2018.00005

[54] 'iCub firmware,' Istituto Italiano di Tecnologia, Accessed: 8th Nov. 2026. [Online]. Available: https://github.com/robotology/icub-firmware

[55] B. Oeyen, *Racket implementation of the Remus virtual machine*, Personal communication, 2026.

[56] P. Weisenburger, J. Wirth and G. Salvaneschi, 'A survey of multitier programming,' *ACM Computing Surveys*, vol. 53, no. 4, pp. 1–35, Sep. 2020, ISSN: 0360-0300. DOI: 10.1145/3397495

[57] 'ScalaLoci – a programming language for distributed applications,' Accessed: 9th Nov. 2026. [Online]. Available: https://scala-loci.github.io/